Ablation: Design Document

Course: CS 5610 Web Development, Northeastern University, Fall 2026
Team: Mayank Devangkumar Tamakuwala, Varshith Sai Pothula
Repository: https://github.com/MayankTamakuwala/Ablation

---

1. Project Description

The problem

Training a model is not the hard part any more. Frameworks are good, checkpoints are free, and a graduate student can launch a fine tuning job in an afternoon. The hard part arrives three weeks later, when a number in your draft needs a provenance.

The number is 91.4. It is in your results table. You remember it was a good run. You do not remember the learning rate, you are not sure whether it used the base checkpoint or the large one, and the only artifact that survives is a file called `final_v2_REAL.pt`. The notebook that produced it has been re-run eleven times since. The terminal scrolled. The Weights and Biases project exists but you stopped logging to it in week three because the SDK broke on the cluster and fixing it was not the thing you were being paid to do.

So you do one of three things. You rerun the experiment and hope it lands in the same place, which costs a day of compute and often does not. You quietly weaken the claim in the paper. Or you report the number and hope nobody asks.

This is not a tooling gap at the top end. MLflow and Weights and Biases are excellent and they solve this problem for teams with the discipline to instrument everything up front. It is a gap at the point where most graduate work actually happens: a single student, a shared cluster, a deadline, and no appetite for configuring infrastructure before running the thing they want to run.

The solution

Ablation is a log, not a trainer. It does not touch your GPU, it does not wrap your training loop, and it has no SDK to break. You tell it what you ran and what you got, and it keeps that record in a form you can query later.

It does four things:

1. **Forces the question to exist before the runs do.** Every experiment carries a research question in its own required field. Not a title, not a tag, a sentence

ending in a question mark. An experiment that cannot state what it is trying to find out is usually a pile of runs with no control, and the field is there to make that visible at the moment of creation rather than at the moment of writing up.
2. **Anchors every experiment to a real checkpoint.** The base model is chosen from a catalog of 1,000 real models pulled from the Hugging Face Hub, so the record says `meta-llama/Llama-3.1-8B-Instruct` rather than "the llama one". Typed model names drift, and two spellings of the same checkpoint make a comparison unverifiable.
3. **Separates the question from the attempts.** Experiments hold the intent, runs hold the measurements. One experiment, many runs. This is the structure that lets you ask whether a comparison was controlled, because a controlled comparison is a set of runs under one question where exactly one field differs.
4. **Keeps the failures.** A run carries a `reproduced` flag, and experiments can be marked abandoned rather than deleted. The dead ends are the most expensive knowledge in a research project and the first thing every other tool throws away.

Scope for Project 3

Three MongoDB collections, two of them with full CRUD, a React client with hooks, an Express JSON API, and Passport authentication, deployed as a single Vercel project.

In scope	Deliberately out of scope
Experiments with full CRUD, owned per user	Side by side controlled comparison view
Training runs with full CRUD, owned per user	Archive and restore, as distinct from delete
Shared catalog of 1,000 real base models	Export to CSV or a paper appendix
Real accounts with hashed passwords	Tags and saved filters
Over 1,000 synthetic records	Charts and metric curves
Status filtering, search, sorting	Teams, sharing, or any multi user feature

The right hand column is not a list of things we ran out of time for. The final project is a direct iteration on this codebase, and spending the whole feature surface in November

leaves nothing to iterate on in December. The comparison view in particular is the most interesting idea in the concept, and it is being held back on purpose.

Why this is useful

The honest test for a course project is whether the authors would use it outside the course. Both of us run training jobs. One of us is a graduate research assistant and a teaching assistant for a graduate course on AI assisted coding; the other has the same problem from the coursework side.

The specific moment this is built for is the one where an advisor asks "is that comparison controlled" and the honest answer is "I think so". Ablation makes that answerable in about four seconds, because the question, the base model and the hyperparameters are all one record away from each other. No other property of the tool matters as much as that.

2. User Personas

Persona 1: Priya Raghunathan, the thesis writer

> "I have a results table with nine numbers in it and I can account for maybe four."

Age and role	25, second year master's student in computer science
Context	Thesis on parameter efficient fine tuning. Defends in eleven weeks
Technical comfort	High. Writes PyTorch daily, has no patience for configuring tools
Devices	Laptop, plus a phone for checking whether a job finished
Volume	Roughly 200 fine tuning jobs over four months

Goals

- Account for every number that appears in her thesis
- Find the exact configuration behind her best result without rerunning it
- Tell the difference between a real improvement and seed noise

Frustrations

- Her results spreadsheet stopped being updated in week three and she knows it
- Three runs in her notes are all called `test`
- She has twice reported a number she could not reproduce afterwards

How Ablation fits into her week

Priya logs a run immediately after it finishes, from the terminal she is already looking at, in under a minute. That is the only behaviour the tool actually demands of her, and it has to stay under a minute or she will stop. The payoff comes in week ten, when she is assembling the results table and needs to put a configuration next to each number. The field she depends on most is the seed, because the question she is always really asking is whether the gap between two numbers is larger than the gap between two seeds of

the same configuration. If she had to give up every field but one, she would keep the hyperparameters and rebuild the rest around them.

Persona 2: Marcus Oyelaran, the inheritor

> "There are forty checkpoints in this drive and no note explaining any of them."

Age and role	26, first year PhD student
Context	Inherited a project from a student who graduated in August
Technical comfort	High, but new to this lab's conventions
Devices	Laptop and a cluster login

Goals

- Reconstruct what the previous student actually tried before building on it
- Record what he learns as he learns it, so the next person is not in his position
- Separate the dead ends from the unfinished threads

Frustrations

- Checkpoint filenames encode a versioning scheme nobody documented
- He cannot tell whether an approach was abandoned because it failed or because the student ran out of semester
- Re-deriving someone else's negative results costs him weeks of compute

How Ablation fits into his week

Marcus is the only persona who writes mostly into the notes field and the status field rather than the metrics. He is doing archaeology: creating an experiment per reconstructed thread, marking most of them `abandoned`, and leaving a sentence saying why. His work looks like data entry rather than research, which is exactly why it never gets done without a place to put it. The status field matters more to him than to anyone else, because `abandoned` with a note is a finding, and a deleted row is not.

Persona 3: Dr. Wen Chen, the principal investigator

"Did you change one thing, or did you change three things and get a better number?"

Age and role	44, associate professor, runs a lab of six
Context	Weekly one on ones with each student. Reviews results she did not produce
Technical comfort	High, but she has not personally launched a job in two years
Devices	Laptop, and a projector in the group meeting room

Goals

- Tell at a glance whether a comparison her student is presenting is controlled
- See where each student actually is without asking them to prepare a slide
- Catch an uncontrolled comparison before it reaches a submission

Frustrations

- Students present their best number, not their comparison
- She asks the same question every week and gets a confident answer that is sometimes wrong
- Reviewing six students' work means six different spreadsheet formats

How Ablation fits into her week

Dr. Chen is the persona who most wants the feature we are deliberately not building in this iteration. What she needs is two runs side by side with the differing fields highlighted. In Project 3 she gets the ingredients: a sortable run table and a filterable experiment list, so she can at least find the two runs and read them next to each other. She is in this document because designing her out now would mean retrofitting her in December, and the data model has to support her question from the first commit even though the view that answers it comes later.

Persona 4: Sam Whitfield, the first timer

> "I called every run `test` and now I have eleven of them."

Age and role	22, senior, first machine learning course
Context	Running systematic experiments for the first time, for a course project
Technical comfort	Moderate. Follows tutorials closely, improvises reluctantly
Devices	Laptop, Colab

Goals

- Not lose the result he got on Tuesday
- Understand what a controlled comparison is by being made to construct one
- Hand in something that looks like a methodology section

Frustrations

- Every run overwrites the last one, in the filesystem and in his notes
- He does not yet know which fields will matter later, so he records none of them
- Professional tools assume a vocabulary he does not have yet

How Ablation fits into his week

Sam is the reason the research question is a required field rather than an optional one. He is not being taught by a tooltip; he is being taught by a form that will not submit until he has written down what he is trying to find out. Every required field in the create form is a decision about what a beginner should be made to think about, and the cost of that choice falls entirely on him. He is also the persona most likely to abandon the tool if the form is long, which is why the create form has six fields and not twelve.

3. User Stories

Stories are split so the two of us build independent collections. Each of us writes the full stack for our own stories: the MongoDB collection, the Express routes, and the React components. Neither set depends on the other being finished, and neither of us calls the other's routes.

Mayank Tamakuwala, the `experiments` collection

Story 1: Defining an experiment before running anything

Sam has a course project due in two weeks and a vague plan to "try some different settings on BERT". He opens Ablation, and the form will not let him past the second field without writing a question. He types "Does freezing the first six layers change accuracy or only training time?", picks `google-bert/bert-base-uncased` from the catalog by typing "bert" into the picker, sets the dataset to `sst2`, leaves the status at planned, and saves. He has not run anything yet. He has a record that says what he is about to find out, which is the artifact his methodology section will be built from.

Acceptance criteria

- Title, research question, dataset and base model are all required
- The base model is selected from the catalog, not typed freehand
- Status defaults to planned, so an experiment can exist before any run does
- A saved experiment appears at the top of the list without a page reload
- Submitting an incomplete form shows which field is missing and does not clear the other fields

Story 2: Seeing where everything stands

Dr. Chen sits down on Thursday before her one on ones. She opens the experiment list and sees every experiment with its status and the question it is asking. She filters to `running` and gets the four that are live right now. She searches `gsm8k` and sees every experiment touching that dataset regardless of status. In ninety seconds she has the agenda for three meetings.

Acceptance criteria

- The list shows title, question, dataset, base model, status and last updated
- Filtering by any of the four statuses narrows the list
- Search matches against title, question and dataset together
- Filter and search compose, so `running` plus `gsm8k` means both
- An unrecognised status in the query string is rejected rather than silently ignored
- The list is paginated, so a thousand experiments do not arrive in one response

Story 3: Keeping the log honest as it ages

Marcus has spent a morning reading the previous student's checkpoints and now knows that three of the threads he reconstructed were the same idea. He edits one to carry the full picture, changes its status from `planned` to `abandoned`, writes a sentence in the notes explaining the approach never beat the baseline, and deletes the two duplicates. The log now reflects what actually happened rather than what he guessed on Monday.

Acceptance criteria

- Every field of an experiment can be edited, including the base model
- Editing reuses the create form rather than opening a separate screen
- Changing status does not require resending the whole record
- Deleting asks for confirmation naming the experiment
- `updatedAt` moves on edit, so the list can sort by genuine recency

Varshith Sai Pothula, the `runs` collection

These three stories belong to the other half of the project. They are stated here because the document describes one application, but the implementation, the acceptance criteria and the mockups for these screens are his to own and refine.

Story 4: Logging a run while the terminal is still open

Priya's job finishes at 11:40pm with a validation accuracy of 0.914. She opens the run form, types the experiment label, the learning rate, batch size, epochs and seed from her command line, pastes the accuracy and loss, marks it reproduced, and saves. It takes her under a minute, which is the only reason she does it at all.

Acceptance criteria

- Hyperparameters, metrics, seed and duration are all captured in one form
- The run records its own copy of the experiment label and base model
- Accuracy is constrained to a fraction between 0 and 1
- A saved run appears in the table without a reload

Story 5: Finding the best result and checking whether it was earned

Dr. Chen sorts the run table by accuracy and looks at the top five. She filters to reproduced runs only, and two of the five disappear. She sorts by learning rate to see whether the survivors share a setting.

Acceptance criteria

- Any numeric column can be sorted ascending or descending
- Runs can be filtered to reproduced or not reproduced
- Sorting is applied in the database, not in the browser, so it holds across pages

- Only an allowed set of fields is sortable, so the query string cannot reach arbitrary document paths

Story 6: Correcting the record

Sam logged a run and typed the learning rate wrong. A different run came from a job that crashed halfway and its numbers are meaningless. He fixes the first and deletes the second.

Acceptance criteria

- Every field of a logged run can be corrected
- Deleting asks for confirmation
- The table reflects the change immediately

4. Design Mockups

Wireframes are low fidelity on purpose. At this stage the questions worth answering are about layout, hierarchy and which controls exist, not about pixels. A sketched style keeps reviewers commenting on structure rather than on border radius.

The screens below cover the `experiments` half of the application. The run table is described in 4.7 at the level the data model requires, but its detailed mockups belong with the stories that own it.

4.1 Sign in

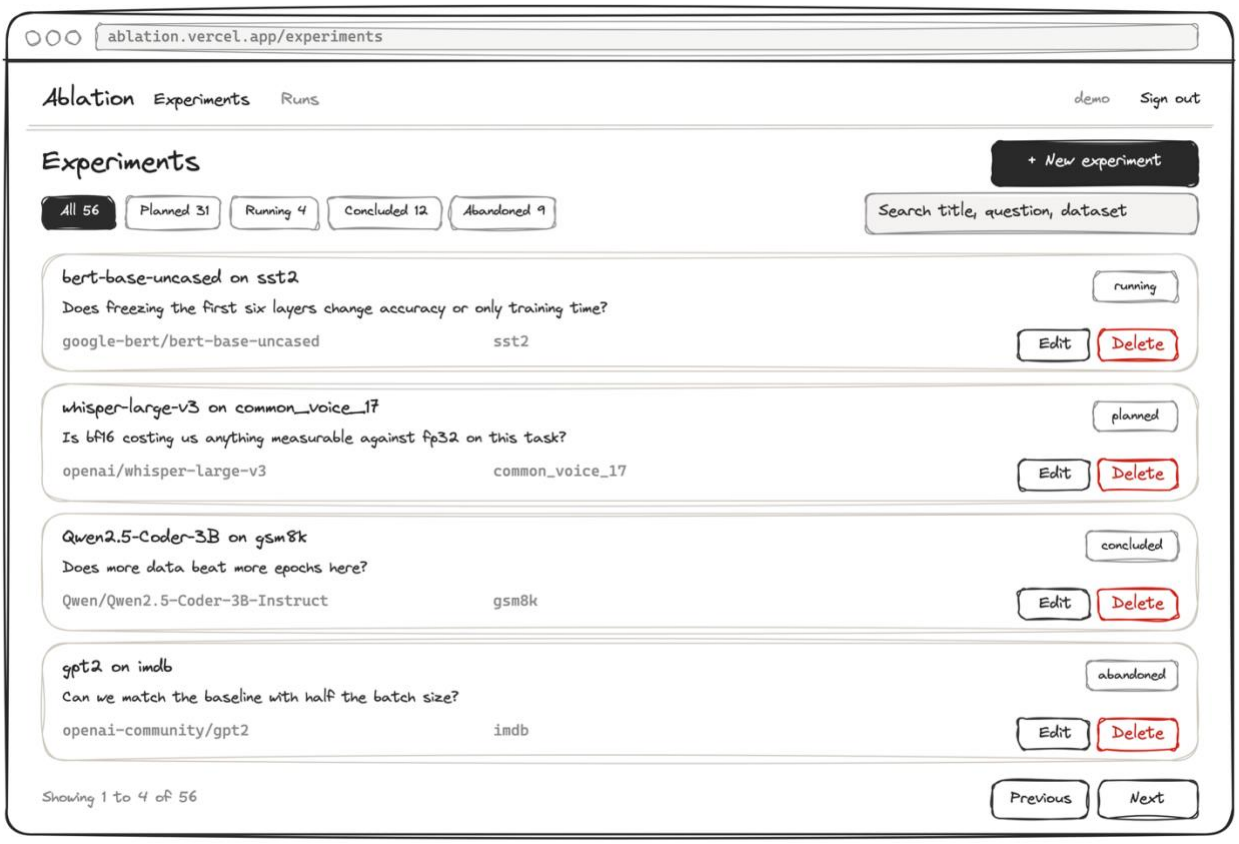


One card, centred, nothing else on the page. Both modes live on the same screen and swap with a single link rather than a route, because an unauthenticated visitor has exactly two possible intentions and making them navigate between two pages to pick one is friction with no purpose.

The demo credentials are printed on the form. This is a course project that will be opened by a grader and by peer reviewers who have no account, and an empty login screen with no way in is a worse failure than a visible shared password. The field order is username, then password, then the submit button, which matches the tab order, which matches the DOM order. Those three agreeing is not an accident and it is what Section 7 is about.

Errors appear above the button, not beside the field, because the server returns one error for the whole credential pair. Attaching "incorrect username or password" to the password field would imply the username was recognised, which is exactly the inference the identical error messages are designed to prevent.

4.2 Experiment list, populated



The page is two stacked regions: controls that are always visible at the top, then the list. The create form is behind a button rather than always open, which is the opposite of the decision made in the previous project. The reason is the record length. A reading list entry was four short fields and belonged inline; an experiment is six fields including a research question and a notes box, and a permanently open form that tall pushes the list itself below the fold on a laptop. The list is what you came for.

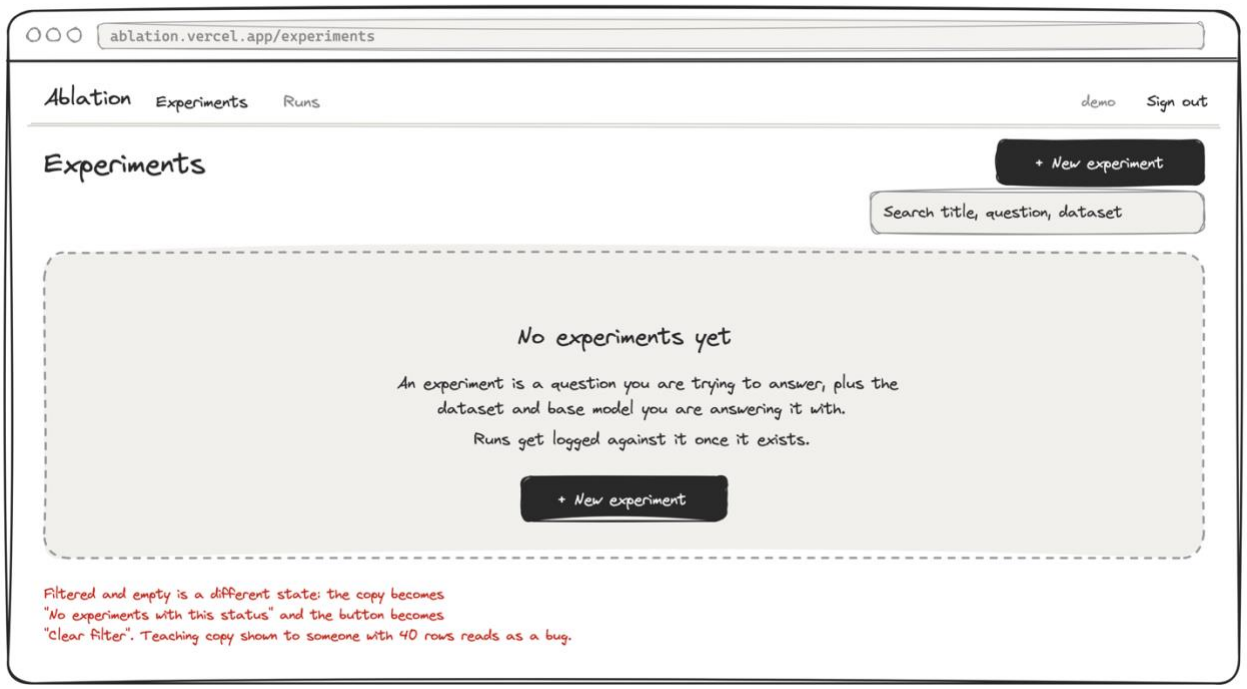
Four things are visible on every row without interaction: the title, the status, the research question, and the base model. The question gets a full line rather than being truncated into a caption, for the same reason the note field got one in the previous project. It is the field the product exists to preserve.

The status filter is a row of toggle buttons rather than a select element. Four options with a count beside each is a case where showing all the options costs less space than hiding them behind a control that has to be opened to be read. The counts are the part

that matters: "31 planned, 4 running, 12 concluded, 9 abandoned" is a sentence about the state of your research that a dropdown cannot say.

Pagination sits at the bottom with the range stated in words. The demo database holds over a thousand experiments, and the list route returns a page rather than all of them.

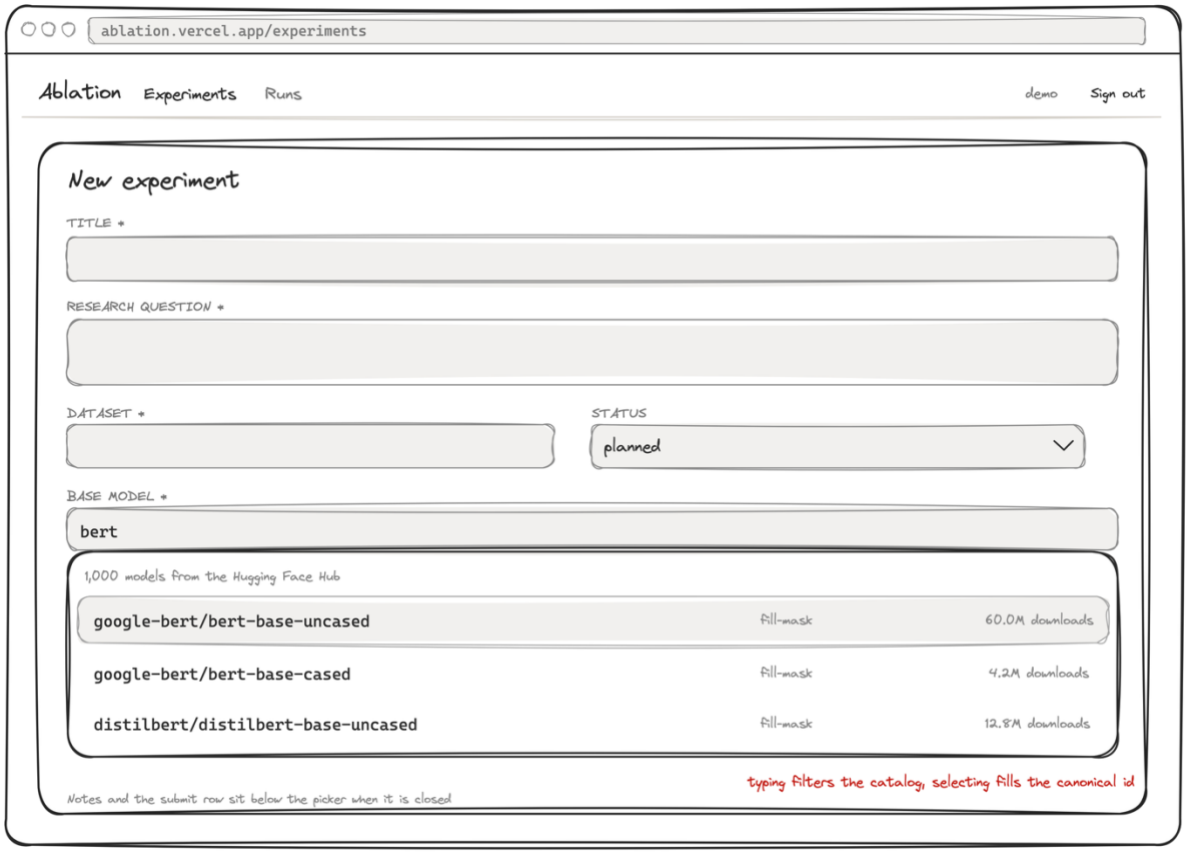
4.3 Experiment list, empty state



Every new account sees this first, so it teaches rather than apologises. The dashed border signals a placeholder rather than a broken region, and the copy points back at the button above it instead of merely announcing that nothing is here.

The empty state has two variants and the distinction matters. With no experiments at all, it explains what an experiment is and offers the create button. With a filter active and no matches, it says "No experiments with this status" and offers to clear the filter instead. Showing the teaching copy to someone who has forty experiments and a narrow filter would read as a bug.

4.4 Create form, with the model picker open

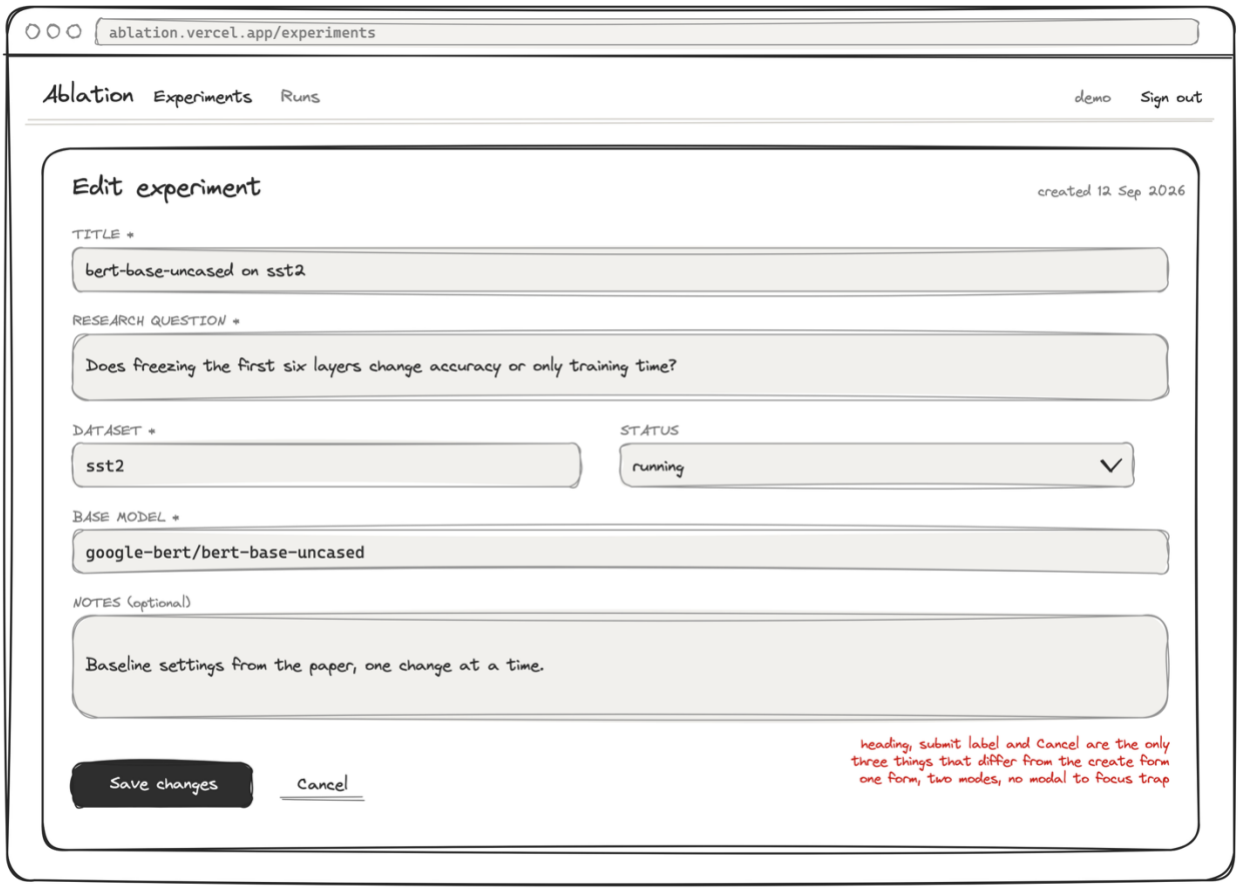


Six fields, in the order a person thinks about them: what is this called, what am I trying to find out, what data, which checkpoint, where is it up to, anything else. Title and question sit at the top because they are required and they are the two that make the record meaningful.

The base model field is a search box over the catalog rather than a select element with a thousand options or a free text input. Typing filters the catalog by name, and each result shows the task and the download count so a choice between two similarly named checkpoints can be made without leaving the form. Selecting one fills the field with the canonical id. Free text here would let two spellings of the same checkpoint into the database and quietly make every comparison involving them unverifiable, which is the failure the whole tool exists to prevent.

The notes field is the only optional one and it is last, so the required path through the form is a straight line with no skipping.

4.5 Edit mode

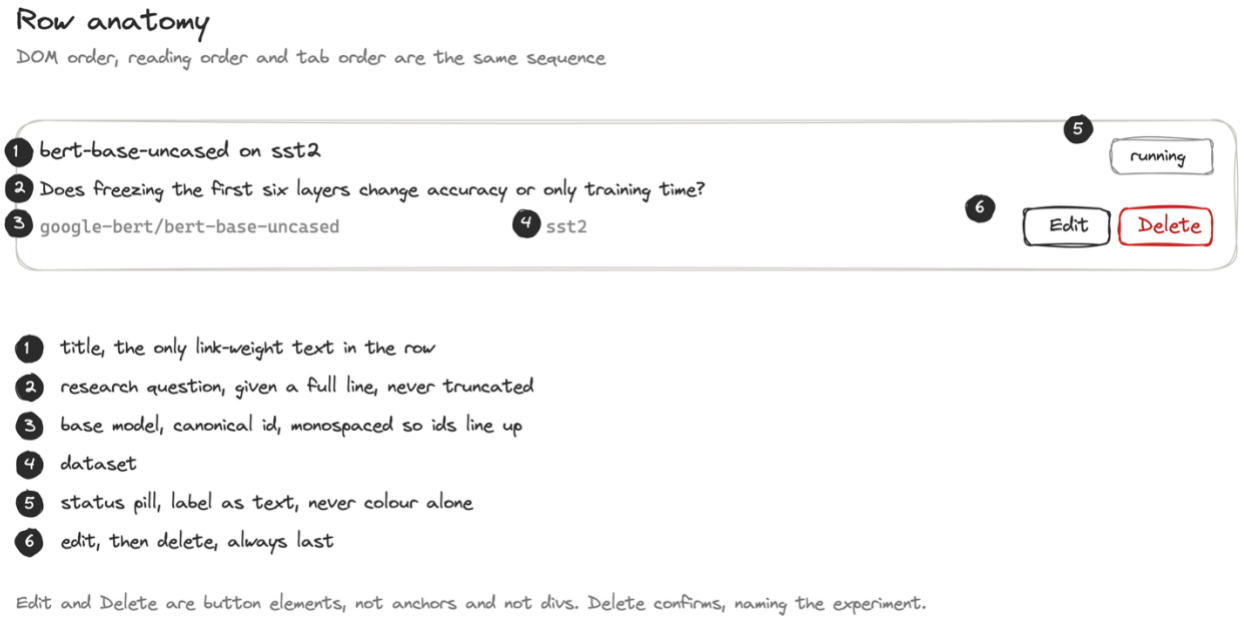


Editing reuses the create form rather than opening a modal or a separate route. One form, two modes. Three things change: the heading becomes "Edit experiment", the submit button becomes "Save changes", and a "Cancel" button appears beside it.

Reusing the form means no focus trap to build, no restore point to manage, and a user who has learned the create flow already knows the edit flow. The cost is that the form cannot be open for a new experiment and an edit at the same time, which is not a thing anyone wants to do.

Status is the one field that can also be changed directly from the list row, without opening the form at all. Moving an experiment from planned to running is a single decision and should cost a single interaction, which is why the API accepts a partial update rather than demanding the whole record back.

4.6 Row anatomy



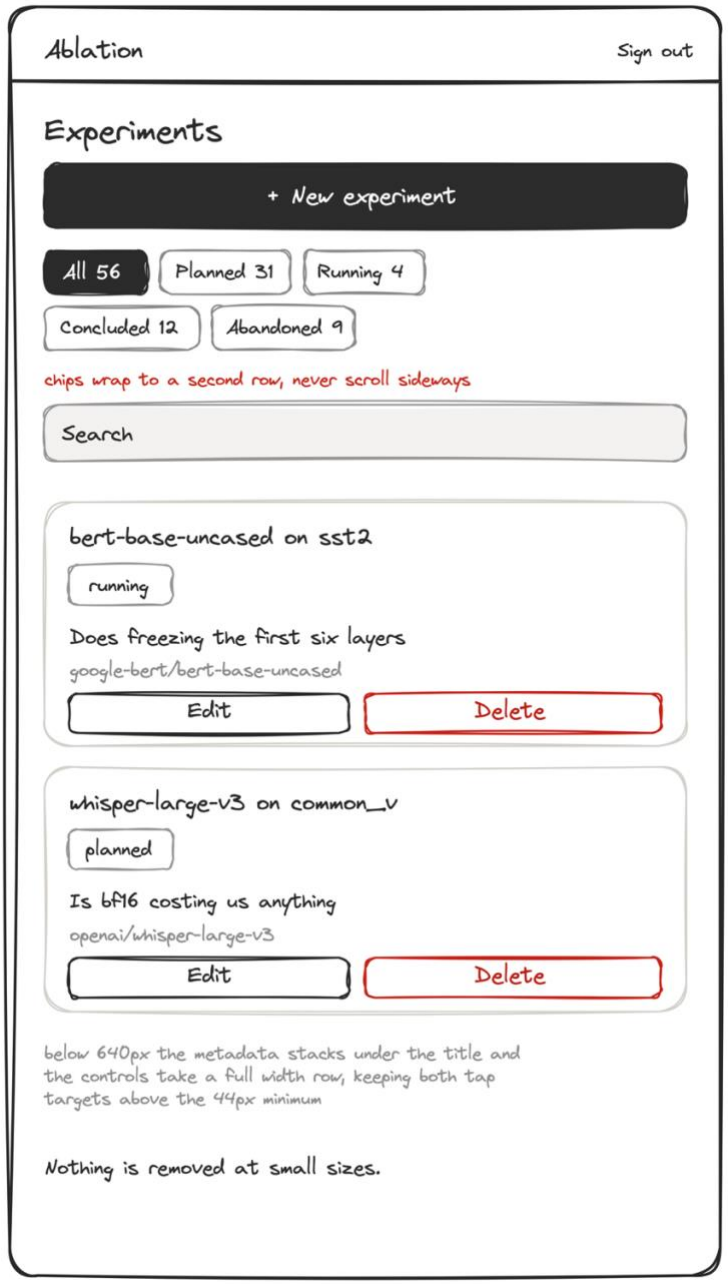
The annotated row, showing what each region is and why it is where it is. Reading order runs title, status, question, base model, dataset, then the controls, which is also the DOM order and the tab order.

The delete control is last in the row and asks for confirmation naming the experiment, because it is the only irreversible action on the page. Edit and delete are both real `button` elements, not anchors styled as buttons and not divs with click handlers. That is worth five points in this project under the non standard tags criterion and ten in the final under keyboard operation, and it costs nothing to do correctly from the start.

4.7 Run table

The run table is the other half of the application: one row per logged run, sortable by any numeric column, filterable by whether the run reproduced. It reads from the `runs` collection only, and carries its own copy of the experiment label and base model, so it renders correctly when the `experiments` collection is empty. Its detailed mockups belong with the stories that own it.

4.8 Mobile layout



Below 720px the form collapses from two columns to one and the status filter chips wrap onto two rows rather than scrolling horizontally, so no option can be hidden off the edge of the screen. Below 640px the row's metadata stacks under the title and the controls move to a full width row beneath it, which keeps both tap targets comfortably above the 44px minimum.

Nothing is removed at small sizes. A layout that hides the status filter on a phone would be hiding the one control that makes the list usable.

5. Data Model

Collections

Three collections. Two carry user work with full CRUD, one is shared reference data that the application only ever reads.

Collection	Contents	Records	Owner	Operations
`models`	Real checkpoints from the Hugging Face Hub	1,000 real	shared reference	read only
`experiments`	A research question and its setup	1,040 synthetic	Mayank	full CRUD
`runs`	One training run and its measurements	synthetic	Varshith	full CRUD
`users`	Accounts	created on registration	shared	create, read
`sessions`	Passport session store, written by `connect-mongo`	transient	library managed	library managed

`models`

Seeded from `https://huggingface.co/api/models`, which is free, needs no key, and returns 1,000 records in a single request. The model id is used as the `_id`, so a re-seed replaces rather than duplicates.

Field	Type	Notes
`_id`	string	The canonical id, for example `openai/whisper-large-v3`
`author`	string	The part before the slash

Field	Type	Notes
`name`	string	The part after the slash
`task`	string	The Hub's `pipeline_tag`
`library`	string	`transformers`, `sentence-transformers`, and so on
`downloads`	number	Drives the default sort
`likes`	number	
`tags`	string[]	Capped at 12
`createdAt`	Date	From the Hub, not from us

127 of the 1,000 models carry no `pipeline_tag`. Those are kept in the catalog but excluded from the synthetic experiment generator, because an experiment pointing at an untagged checkpoint is noise in the picker.

`experiments`

Field	Type	Rules
`_id`	ObjectId	
`userId`	string	Set from the session, never from the request body
`title`	string	Required, 120 characters
`question`	string	Required, 500 characters
`dataset`	string	Required, 120 characters

Field	Type	Rules
`baseModel`	string	Required, 160 characters, chosen from the catalog
`status`	enum	`planned`, `running`, `concluded`, `abandoned`. Defaults to planned
`notes`	string	Optional, 2,000 characters
`createdAt`	Date	Set on insert
`updatedAt`	Date	Moved on every update

`runs`

Owned by the other half of the project. The field that matters to this document is the denormalisation: a run stores its own copy of `experimentLabel` and `baseModel`, captured when the run is logged.

That copy is the reason the two halves are independent. The run table never joins against `experiments`, so it renders with zero experiments in the database, and the experiment list renders with zero runs. Neither of us can block the other, and neither of us calls the other's routes. The tradeoff is the usual one for denormalisation: renaming an experiment does not rewrite the label on runs already logged. For a log of what happened, that is arguably correct behaviour rather than a defect, since the run was logged against the name the experiment had at the time.

API routes

Method	Route	Purpose
`GET`	`/api/health`	Liveness, used to check a deployment
`GET`	`/api/auth/me`	The signed in user, or `null`
`POST`	`/api/auth/register`	Create an account and sign in
`POST`	`/api/auth/login`	Sign in

Method	Route	Purpose
`POST`	`/api/auth/logout`	Destroy the session
`GET`	`/api/models`	Catalog, filtered by search and task
`GET`	`/api/models/tasks`	Distinct tasks, for the picker's filter
`GET`	`/api/experiments`	The caller's experiments, filtered and paginated
`POST`	`/api/experiments`	Create
`GET`	`/api/experiments/:id`	One experiment
`PUT`	`/api/experiments/:id`	Update, accepts a partial body
`DELETE`	`/api/experiments/:id`	Delete
`GET`	`/api/runs`	The caller's runs, sorted, filtered, paginated
`POST`	`/api/runs`	Create
`GET`	`/api/runs/:id`	One run
`PUT`	`/api/runs/:id`	Update
`DELETE`	`/api/runs/:id`	Delete

Everything except `/api/health` and the three auth routes requires a session.

Ownership and isolation

Every query against `experiments` and `runs` filters on `_id` and `userId` together, and `userId` comes from the session rather than the request. A request for another account's record returns 404 rather than 403, because 403 confirms the record exists and turns a guessable id into an enumeration oracle.

Input rules

Rule	Behaviour
Whitespace around a value	trimmed silently
A missing required field	rejected with the field named
A value past its length limit	rejected with the limit stated
A status outside the four	rejected, including in the query string
An update naming no known field	rejected rather than stamping `updatedAt`
A malformed `ObjectId`	404, checked before the query is built
Regex characters in a search box	escaped, so `.*` matches nothing
A non numeric `limit` or `skip`	falls back to the default, then clamps to a maximum

The split is deliberate. A trailing space is a slip the application should absorb quietly. A status of `bogus` is a client bug, and silently ignoring it would mean returning an unfiltered list to a caller who believes it is filtered.

Indexes

Collection	Index	Serves
`users`	`{ username: 1 }` unique	Login, and duplicate registration
`models`	`{ downloads: -1 }`	The catalog's default sort
`models`	`{ task: 1 }`	The picker's task filter

Collection	Index	Serves
`experiments`	`{ userId: 1, updatedAt: -1 }`	The default list query
`experiments`	`{ userId: 1, status: 1 }`	The status filter and its counts

6. Visual Design

The look is chosen now and carried into the final project unchanged. Retrofitting typography and a palette is more expensive than committing to them early, and the visual decisions are worth thirty points in December.

The reference is a laboratory notebook: warm paper, dark ink, one accent used sparingly, and numbers set in a face where a column of figures lines up.

Typography

Role	Face	Rationale
Headings	Space Grotesk, 500 and 600	A geometric sans with enough character to carry a heading at 20px without being set large. Its tighter apertures read as technical rather than friendly, which suits a tool for recording measurements
Body and UI	Inter, 400 and 500	Designed for screen UI at small sizes. Its tall x-height keeps a 500 character research question legible at 15px
Metrics and hyperparameters	IBM Plex Mono, 400	Tabular figures. A learning rate of `0.0000283` next to `0.0003` only reads as different magnitudes when the digits align vertically

Three faces is one more than a page like this usually needs. The third earns its place because the entire product is about comparing numbers, and proportional figures in a metrics column defeat the comparison the user came to make. All three are on Google Fonts, none is a browser default, and the stack falls back to `system-ui` so a blocked font request degrades rather than breaks.

Scale, in rem against a 16px root: 2.0 page title, 1.25 section heading, 1.0 body, 0.9375 secondary, 0.8125 pill and caption. Five steps, nothing between them.

Palette

Every pair below was measured, not estimated.

Token	Hex	On	Ratio	Use
`--ink`	`#14181F`	paper	**17.06**	Body text, headings
`--ink-muted`	`#5A6472`	paper	**5.75**	Labels, metadata, captions
`--paper`	`#FBFAF8`			Page background
`--surface`	`#FFFFFF`			Cards and rows
`--line`	`#E3E0D8`	paper	1.26	Decorative hairlines only
`--line-control`	`#8A8578`	paper	**3.53**	Input and button borders
`--accent`	`#2B50D8`	paper	**6.21**	Primary action, links, focus ring
`--on-accent`	`#FFFFFF`	accent	**6.48**	Primary button label
`--danger`	`#B3261E`	paper	**6.27**	Destructive action, error text
`--on-danger`	`#FFFFFF`	danger	**6.54**	Delete button label

Every text pair clears WCAG AA at 4.5:1. `--line` is the one value that does not, at 1.26, and it is restricted to decorative hairlines between rows where nothing depends on seeing it. Anything that is the boundary of a control uses `--line-control` at 3.53, which clears the 3:1 that axe enforces for non text contrast. Getting this wrong is the single most common way a project fails that audit, and it fails on input borders specifically.

Approve and cancel are fixed across the application. Primary and confirming actions are `--accent` filled with white text. Destructive actions are `--danger`, outlined in the list row and filled only inside a confirmation dialog. Cancel is always a plain text button with no fill, so it never competes with the action it sits beside. The final project grades consistency here explicitly, so the rule is written down rather than rediscovered per component.

Status colours

Status	Hex	On white	Reading
`planned`	`#64748B`	**4.76**	Slate. Nothing has happened yet
`running`	`#0F766E`	**5.47**	Teal. Live, the only status that is currently costing compute
`concluded`	`#1E5128`	**9.28**	Green. Answered, whichever way it went
`abandoned`	`#7C5A3C`	**6.20**	Brown. Stopped, deliberately, and kept on purpose

`abandoned` is brown rather than red on purpose. Red would read as an error, and abandoning an approach because it did not work is a result, not a failure. Marcus's entire workflow consists of creating records in this status.

Status is never communicated by colour alone. Every pill carries its label as text, so the information survives a monochrome print, a colour vision deficiency, and a grader reading a screenshot.

Spacing

A 4px base, used at 4, 8, 12, 16, 24, 32 and 48. Card padding is 16 on mobile and 24 above 720px. The list gap is 12. Nothing uses a value off this scale, which is what makes alignment auditable rather than a matter of opinion, and alignment is ten points in the final.

7. Accessibility Decisions

These are decisions made now, in Project 3, for points awarded in the final. Keyboard operation and a clean axe pass are twenty points there, and both are close to free if the markup starts correct and close to a rewrite if it does not.

Native elements for everything. Buttons are `button`, the status filter is a group of `button` elements with `aria-pressed`, text inputs are `input`, the notes field is `textarea`, and the status selector is a `select`. There is no component in this design that needs a custom widget, which is the reason the concept was chosen over anything involving a map, a canvas or drag and drop. Native elements arrive with focus handling, keyboard behaviour and an accessibility tree already attached.

DOM order is reading order is tab order. No `tabindex` above 0 anywhere. Where the visual layout places something out of document order at a breakpoint, the layout changes rather than the markup.

Landmarks and headings. One `h1` per page, `header`, `main` and `nav` used once each, the experiment list as a `ul` of `li` rather than a stack of divs, and the form wrapped in a real `form` element that submits on Enter.

Labels, not placeholders. Every input has a `label` with a `for` attribute. A placeholder disappears the moment a user starts typing, which is exactly when someone who needed the hint still needs it.

Focus is visible and deliberate. A 2px `--accent` outline with a 2px offset, so the ring sits on the page background rather than on the control. On a filled accent button the offset is what makes the ring visible at all, since a ring the same colour as the fill has a contrast ratio of 1.00 against it.

Errors are announced, not just coloured. Validation messages are text, in `--danger`, in a container with `role="alert"`, positioned so the message and the control it refers to are adjacent in the DOM.

Confirmation before anything irreversible. Delete names the experiment it is about to remove. The confirmation is a native `dialog`, so focus trapping, Escape to close and inert background come from the browser rather than from code we would have to write and then test with a screen reader.

Motion is minimal and respects the preference. The only transitions are 120ms colour changes on hover and focus, wrapped in `prefers-reduced-motion`.